

This homework has 8 questions, for a total of 100 points and 5 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

For bonus/extra-credit questions, we will provide very limited guidance in office hours and on Piazza, and we do not guarantee anything about the difficulty of these questions.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up*.

(0 pts) 0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: Shikha Nukala, desigirl

(10 pts) 1. **Self assessment.**

Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't turn in the previous homework, then (1) state that you didn't turn it in, and (2) pick a problem that you think is particularly challenging from the previous homework, and explain the answer in your own words. You may reference the answer key, but your answer should be in your own words.

Solution: HW8 Question 4b

Prove that the reduction is correct by proving the following, for any 3CNF formula φ :

- $\varphi \in 3SAT \implies (A, s) \in SUBSETSUM$;
- $(A, s) \in SUBSETSUM \implies \varphi \in 3SAT$.

Solution:

To prove that the reduction is correct, we can start with $\varphi \in 3SAT \implies (A, s) \in SUBSETSUM$. If we assume that φ is satisfiable in 3SAT, then there exists a truth assignment that satisfies φ . For each clause c_j we know that there are integers a_j and b_j where the j th digit is 1 and all other digits are 0 by the reduction. Then we have integers t_i and f_i such that the $(m+i)$ th digit is 1 if the condition for each variable x_i appears in c_j . Knowing these, we know that c_j contributes to the sum because there is a j th digit of 1 for integers in c_j and that each variable contributes to the sum through the 1s in t_i and f_i . The target sum has first m digits being 3 and last n digits as 1, so A is constructed of all these

numbers, and from the example above we know that it is possible to select a subset out of A such that the sum of elements in the set equals s , and we know that there can be a sum of 3 in every m digits because each digit impacts the sum at least once where applicable.

This solution is incorrect because in the first direction it must be proven that any satisfiable formula in 3SAT maps to a set A . Since there are three t/f values in the example in the problem, we can see that if A satisfies the assignment of the current function, then there must be at least one value of those three true values to be true. Therefore each of the m most significant digits being chosen with either a or b can be made to equal exactly three. From the other direction, the subset of A that sums to s must yield a satisfying assignment to the original formula. Therefore, to prove this, we know exactly one false or true value must be in the subset of n least-significant digits. Therefore since each clause of the function is an or statement of three literals as shown in the problem, each clause must amount to a truth value which makes this subset a viable function in 3SAT. The original solution was missing these steps in each direction proof which are required to complete the proof that one set example leads to another.

(12 pts) 2. **Undecidable vs. NP-hard vs. NP-complete.**

Show that the undecidable language L_{ACC} is NP-hard, but is not NP-complete.

You may use the fact that $NP \subseteq EXP$ without proof. (This was the result of the extra-credit problem from HW7. Recall that EXP is the class of all languages that are decidable in exponential time, i.e., in time $O(2^{n^k})$ for some constant k where n is the input size.)

Solution: First we must show that the undecidable language L_{ACC} is NP-hard. To show this we use a proof. Since we know the Boolean formula SAT is NP-hard, we will show a reduction from SAT to L_{ACC} .

First for the reduction we can use a Boolean formula B to construct a directed graph G :

- 1) for every clause in B , create a vertex vc
- 2) for each literal in B , create two vertices opposite to each other vl, vnl .
- 3) for each clause containing a literal, add an edge from the vertices vc to vnl .
- 4) for each clause containing the opposing literal, add an edge from vc to vl .
- 5) for each pair of literals that cannot both be true at the same time add an edge from $vnl1$ to $vl2$ and $vnl2$ to $vl1$.
- 6) these additions should create a Hamiltonian cycle, and the function in SAT B is only satisfiable if G contains a Hamiltonian cycle.

We can see this reduction runs in polynomial time as it takes linear time to add each edge to the graph by going through every clause and literal in the function B .

Therefore if B is satisfiable, then we can construct a Hamiltonian cycle in G . For each clause in B that is satisfied by an assignment in B , select a literal in that clause that is true and follow the edge from the vertex of the clause to the literal vertex. Since each clause in a SAT function is satisfied by an assignment to B , there can be a path that covers all vertices from each clause and from each literal that is true. We also see that a path is constructed

to all complementary literals that are not true therefore the path is not crossed between the two. Finish the cycle by adding an edge between each last literal vertex to each beginning clause vertex.

If G has a Ham cycle in it, then there can be a SAT function B constructed. First each clause is covered by at least one literal, and each complementary literal is not selected due to edges between them, so at the start of the edges we assign the clauses and the remaining vertices we assign the literals.

Therefore SAT is shown to reduce to L_{ACC} in polynomial time which means L_{ACC} is also NP -Hard.

To show L_{ACC} is not NP -Complete, since we know $NP \subseteq EXP$, if L_{ACC} were NP -Complete then it would have to be in NP and therefore would have to be decidable in some exponential time as it runs in. Since we know L_{ACC} is undecidable then there is no possibility that L_{ACC} can be in NP , therefore it cannot be NP -Complete.

(16 pts) 3. **Search vs. decision: any Hamiltonian path.**

Define the language

$$\text{ANYHAMPATH} = \{G : G \text{ is an undirected graph with a Hamiltonian path}\}.$$

Suppose that there exists an efficient algorithm D that decides ANYHAMPATH. Give an efficient algorithm that, on input an undirected graph G , outputs a Hamiltonian path in G if one exists, otherwise it outputs “No Hamiltonian path exists.” Prove that your algorithm is correct and runs in polynomial time.

Solution: To show an algorithm to find any Hamiltonian path in G being an undirected graph G , we can show:

if we run algorithm D on an undirected graph G :

- 1) if D returns that there does not exist a path then the algorithm halts on the input and outputs there does not exist a path
- 2) if D returns there does exist a path, then identify any vertex in G called v
- 3) start from v and go through G using an algorithm such as breadth-first search to find a Hamiltonian path in G
- 4) output the path that is found and halt the instance.

From this algorithm layout, we can see that there is a decision on any input, since if there does exist a path then the path is returned by running in polynomial time using a search function while if a path does not exist then the algorithm simply halts anyway. Therefore on all valid inputs the algorithm will output and halt.

We know this is polynomial time complexity because the 3rd step of the algorithm takes polynomial time as the polynomial function of the input size binds the time. The other steps are decision steps and therefore also take polynomial time, therefore the algorithm runs in polynomial time.

(20 pts) 4. **The fire-station problem.**

The state government is building fire stations in a region of Northern Michigan that has many small towns connected by roads. Since it's expensive to install a fire station in every town, the government has decided that it wants to build as few fire stations as possible, so that each town either has a fire station, or is directly connected by a road to a neighboring town that has a fire station.

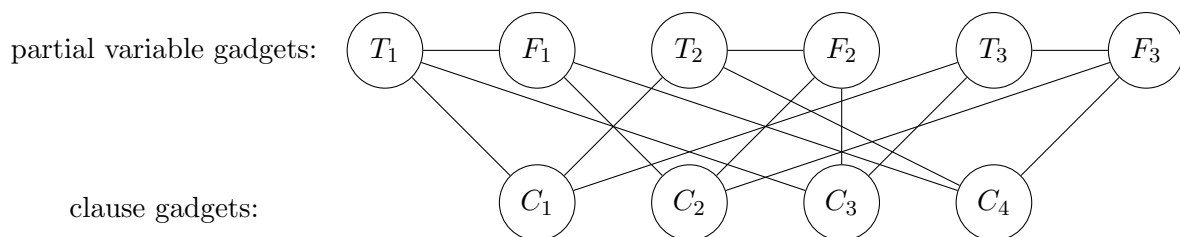
Formally, the decision problem FIRESTATION is defined as follows: given an undirected graph $G = (V, E)$ and a positive integer “budget” k , does there exist a subset $S \subseteq V$ of at most k vertices such that for every $v \in V$, either $v \in S$ or there is an edge $(u, v) \in E$ such that $u \in S$? (Make sure to understand how this problem differs from the similar-looking VERTEXCOVER problem!) In this problem you will show that FIRESTATION is NP-hard, by proving that $3\text{SAT} \leq_p \text{FIRESTATION}$.

Since FIRESTATION has some resemblance to VERTEXCOVER, a good source of inspiration is the 3SAT-to-VERTEXCOVER reduction from lecture. To start, we modify the reduction by making each “clause gadget” a *single vertex* (rather than a triangle of three vertices). To

compensate for the simpler clause gadgets, the “variable gadgets” will need to be a little more complex, in a way that you will determine. In addition, the “budget” k will need to be different. As an *incomplete* example of the modified reduction, consider the Boolean formula

$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3).$$

The reduction starts by building the following *partial* graph; you will define appropriate other nodes and edges for the variable gadgets:



Show the following two things:

1. that FIRESTATION is NP-hard, by proving that $3SAT \leq_p FIRESTATION$;
2. a diagram depicting the full output of your reduction for the formula φ given above.

As always, your proof will need to include all of the steps outlined in Handout 3: NP-Hardness Proofs.

Solution: First to prove FIRESTATION is NP-hard, we need to prove the reduction $3SAT \leq_p FIRESTATION$. We can start by outlining a reduction from 3SAT to FIRESTATION. First if we have an instance of 3SAT called formula B, then we construct an instance of FIRESTATION as:

- 1) for each literal x_i in B, we can create two vertices for them called t_i and f_i to represent the true or false assignment of the literal.
- 2) for each clause in B, we can create one vertex called v_i
- 3) we can connect each clause v_i to its corresponding literal(s) by connecting it with edges to each t_i and f_i that corresponds. Then connect every complementary vertex pair to each t_i and f_i to each other so they are not connected to the actual values.
- 4) for the fire station budget set k equal to the number of clauses in B such that k vertices corresponds to all clauses in B.

First we can see that this instance of the reduction is polynomial time running. To connect each clause to each literal the program runs linearly through each clause then linearly through each literal, therefore is at most exponential time of $O(n^2)$ and satisfies the polynomial time requirement.

To prove if B is satisfiable then there is a solution to FireStation with at most k vertices, we can show through the reduction. If B is satisfiable, then there is an assignment of true and false values to the literals so that each clause in B is true. For each clause v_i if we select a literal x_i that evaluates to true under the assignment, then we include the corresponding

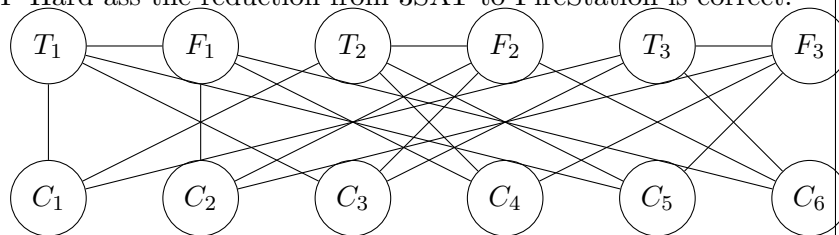
vertex t_i to the set of the solution vertices of FireStation S . Since each clause is true, each vertex that represents a clause will be connected to at least one vertex in S that is a literal value corresponding to the truth assignment, so each vertex in the graph is either in S or connected to a vertex in S . Therefore there does exist a subset S of at most k (B clauses) vertices such that there is an edge for every vertex with one vertex being in S .

Next we have to show if there is a solution to FireStation that has at most k vertices then the corresponding boolean formula B is satisfiable. If there is a solution set S with at most k vertices, then for each v_i , there must be a connected edge such that one vertex in the edge is in S . Therefore we can set each vertex v_i as a clause in the formula, then if there is a vertex connected that is in S from the current clause vertex, change the connected vertex to a truth value t_i or f_i that corresponds to a literal in B . Do this for all edges in the graph until every vertex is checked and either remained as a clause vertex or converted into a literal vertex in S or a complementary vertex without connection to S . Since this can be done for any undirected graph with at most k vertices corresponding to clauses in B , this is a valid satisfiable formula.

Therefore FireStation is NP-Hard as the reduction from 3SAT to FireStation is correct.

partial variable gadgets:

clause gadgets:



5. Approximate knapsack.

Recall the 0-1 *knapsack* problem, in which we are given a set of items having weights and values, and wish to select a subset of the items so that their total weight does not exceed a specified capacity, and their total value is maximized.

More specifically, an instance is a vector of weights $W = (W_1, \dots, W_n)$, a vector of values $V = (V_1, \dots, V_n)$, and a knapsack capacity C , all of which are non-negative integers. (Without loss of generality, each $W_i \leq C$, because otherwise the i th item cannot be selected, so it is irrelevant.) The desired output is a subset $I \subseteq \{1, \dots, n\}$ of the items that maximizes the total value $\sum_{i \in I} V_i$, subject to the constraint that the total weight $\sum_{i \in I} W_i \leq C$.

Recall from lecture that the Combined-Greedy algorithm for this problem is defined as follows:

1. Run the Single-Greedy algorithm,
2. Run the Relatively-Greedy algorithm,
3. Select one of the outputs that has the most total value (breaking a tie arbitrarily).

(Refer to the lecture for the definitions of the two sub-algorithms.)

In this question, you will prove that the Combined-Greedy algorithm is a $1/2$ -approximation algorithm for the 0-1 knapsack problem.

(8 pts)

- (a) Recall from lecture that one way to prove the correctness of an approximation algorithm for a maximization problem is to show a lower bound on ALG , the value obtained by the

algorithm, and show a related upper bound on OPT , the value of an optimal fractional solution. In this part, you will show an upper bound on OPT .

To do this, it will help to recall from Homework 5 the *fractional* knapsack problem, a variant of the 0-1 problem that allows for taking any *partial* amount of an item. That is, for an item of weight W and value V , we may select some fractional amount $t \in [0, 1]$, which has weight $t \cdot W$ and value $t \cdot V$. Recall that the optimal value for 0-1 knapsack is at most the optimal value for the fractional version (on the same knapsack instance).

Prove that the optimal value OPT for the 0-1 knapsack problem is at most the *sum* of the values obtained by the two sub-algorithms.

Hint: It may help to recall from Homework 5 the greedy algorithm for the fractional knapsack problem, which is closely related to Relatively-Greedy (make sure to understand the difference!). You showed that this “Fractional-Greedy” algorithm obtains an optimal solution for fractional knapsack. You can use that fact here without repeating the proof.

Solution: To prove that the OPT for the 0-1 knapsack problem is at most the sum of the values obtained by the two sub-algorithms, we know that the fractional knapsack problem allowed for fractional amounts of items and therefore was able to be more optimal than the 0-1 algorithm. Therefore the 0-1 knapsack problem is at most (many times less than) the optimal value for the fractional knapsack problem for any input values of the problem for the sets or value. Therefore we can set OPT_1 and the optimal value of the 0-1 problem and OPT_2 as the optimal value of the fractional problem. We need to show that $\text{OPT}_1 \leq \text{OPT}_2$ for any instance of the problem. We know from the previous homework for the fractional knapsack problem that the Greedy algorithm for the problem sorted items by value per unit of weight then selected items using greedy selection until there were either no items left or the knapsack was full. Therefore if we have an optimal solution from that greedy solution, we can say that $\text{OPT}_1 \leq$ the total value obtained by the greedy algorithm for the fractional problem.

For the Single-Greedy algorithm, we know the algorithm selects items based on individual values, and for the Relatively-Greedy algorithm, the algorithm selects items based on relative values per unit weight. Therefore we can see that the total value obtained by the Single-Greedy algorithm should be more than or equal to the total value obtained by the relative greedy algorithm as the single algorithm only selects based on value. Therefore the value of Relatively greedy $\leq$ the value of Single-Greedy. Therefore since the fractional greedy problem is also a relatively-greedy problem, the total value of fractional greedy $\leq$ the total value of single-greedy as well. Therefore we can see that Since $\text{OPT}_1 \leq \text{OPT}_2 \leq \text{Single-Greedy}$, we can see that since OPT_2 is relatively greedy we have the fact that $\text{OPT}_1 \leq \text{relatively-greedy total value} + \text{single-greedy total value}$. Therefore we have shown that the OPT optimal value of the 0-1 knapsack problem is at most the sum of the values obtained by the two sub-algorithms.

- (6 pts) (b) Using the previous part, and by establishing a suitable lower bound on the value ALG obtained by Combined-Greedy, show that it is a $1/2$ -approximation algorithm for the 0-1 knapsack problem.

Solution: We can show that the Combined-Greedy algorithm is a $1/2$ -approximation algorithm for the 0-1 knapsack problem by showing that $\text{OPT} \leq \text{ALG}$ of the 0-1 problem. From the last part we saw that $\text{OPT} \leq \text{relatively-greedy total value} + \text{single-greedy total value}$. Therefore we can say ALG is the total value obtained by the Combined-Greedy algorithm. Since the Combined-Greedy algorithm selected the output of Single or Relatively greedy with the higher total value, we know that $\max(\text{Single Greedy total value}, \text{Relatively Greedy total value}) \leq \text{ALG}$. Now to find the lower bound on ALG , we consider the worst case that Single Greedy and Relatively Greedy have disjoint set solutions. Then we know that the solution to $\text{ALG} = \text{Single} + \text{Relatively total values}$. Since we know that relatively greedy total value $\leq \text{Single greedy total value}$, we know that $2 * \text{relatively greedy total value} \leq \text{Single greedy total value} + \text{relatively greedy total value} \leq \text{ALG}$. From the previous part we can rewrite this as $\text{OPT} - \text{relatively greedy total value} \leq 2 * \text{relatively greedy total value} \leq \text{ALG}$. Then that becomes $\text{OPT} \leq \text{ALG} + \text{relatively greedy total value}$. Since ALG is the max of either sub-algorithm, we take the half of OPT that is a greater total value, therefore we have $1/2 * \text{OPT} \leq \text{ALG}$. Therefore this is the lower bound on ALG and the value obtained by the Combined Greedy algorithm is at least half of the optimal value for the 0-1 knapsack problem.

6. Approximate f -SetCover.

In this question, you will consider a variant of the SETCOVER problem where each element of the universe is in a limited number of subsets. Formally, in the f -SETCOVER problem, we are given a “universe” (set) U and subsets $S_1, \dots, S_n \subseteq U$ where each universe element appears in at most f of the subsets. The goal is to find a smallest collection of the subsets that “covers” U , i.e., an $I \subseteq \{1, \dots, n\}$ of minimum size such that $\bigcup_{i \in I} S_i = U$. We assume that $\bigcup_{i=1}^n S_i = U$, otherwise no solution exists.

You will analyze an approximation algorithm “ f -cover” for the f -SETCOVER problem. The algorithm is a generalization of the “double cover” algorithm for VERTEXCOVER from lecture, and it works essentially as follows: while there is some uncovered element u in the universe, add to the cover *all* the subsets to which u belongs. The formal pseudocode is as follows. The notation $I(u) = \{i : u \in S_i\}$ for $u \in U$, that is, $I(u)$ indicates the subsets to which u belongs.

```

1: function  $f$ -COVER( $U, S_1, \dots, S_n$ )
2:    $I = C = \emptyset$                                 ▷ selected indices  $I$ , covered elements  $C$ 
3:   while  $C \neq U$  do                                ▷ not all elements are covered
4:     choose an arbitrary  $u \in U \setminus C$           ▷ element  $u$  is not yet covered
5:      $I = I \cup I(u)$ ,  $C = C \cup \bigcup_{i \in I(u)} S_i$     ▷ add all subsets  $S_i$  containing  $u$  to the cover
6:   return  $I$ 

```

Fix some arbitrary f -SETCOVER instance, and let I^* denote an optimal set cover for it. Let E denote the set of elements u chosen in Step 4 during an execution of the algorithm, and let I denote the algorithm’s final output.

- (7 pts) (a) Prove that $I(u) \cap I(u') = \emptyset$ for every distinct $u, u' \in E$. In other words, prove that if u and u' are each selected as the uncovered element in different iterations, none of the S_i contains both u and u' .

Solution: We can prove $I(u) \cap I(u') = \emptyset$ for every distinct $u, u' \in E$ through contradiction. If there indeed does exist elements $u, u' \in E$ such that $I(u) \cap I(u') \neq \emptyset$, then there is a subset that contains both u, u' . Without loss of generality, if u is chosen in some iteration p and u' was chosen in some iteration p' such that $p \leq p'$, then for iteration p , the algorithm added all subsets with u to the cover so the index is included in $I(u)$ at the end of iteration p . For iteration p' , the algorithm added all subsets with u' to the cover. Therefore the current index of the subset with u and u' is also added to $I(u')$. But since $p \leq p'$, all subsets with u were already added, so the current index has already been added, which means u' was also added in that subset. But that means that u' is added before iteration p' , which contradicts the assumption of the statement. Therefore, the statement $I(u) \cap I(u') \neq \emptyset$ cannot be true, thus $I(u) \cap I(u') = \emptyset$ is true and the statement is proven.

- (7 pts) (b) We want a lower bound on $\text{OPT} = |I^*|$. Using the previous part, prove that $|E| \leq |I^*|$.

Solution: To prove that $|E| \leq |I^*|$, from the algorithm in each iteration, the algorithm selects an uncovered element u and adds all subsets with u in it to the cover with $I(u)$ being the set of indices of the subsets all containing u . Since we know that $I(u) \cap I(u') = \emptyset$, if we have $I_1 \dots I_k$ as the set of indices of the subsets added to the cover over all iterations, we know that $|E| = |I_1 \cup I_2 \cup \dots \cup I_k|$. Therefore since $|I^*|$ is the set of indices with in the set of the optimal cover, it is a minimum set and must have at least one subset for each element in the universe. So $|U| \leq |I^*|$. Since each subset index corresponds to some distinct element in the universe set and each element is only covered by the function once, we know that $|U| = |I^*|$. Therefore we know that $|E| \leq |I^*|$ as the set of elements chosen in step four is as most as the number of indices iterated over. Therefore $|E| \leq |I^*|$ is proved and $\text{OPT} = |I^*|$ had a lower bound.

- (7 pts) (c) We want an upper bound on $ALG = |I|$. Prove that $|I| \leq f \cdot |E|$, and conclude that the f -cover algorithm is an f -approximation algorithm for the f -SETCOVER problem.

Solution: To prove that $|I| \leq f \cdot |E|$, we know that all subsets containing an uncovered element are added to the cover of I . Therefore if $I(u)$ is the set of indices of subsets containing u , we know that each element in the universe U appears in at most f subsets, therefore we know the size of $I(u) \leq f$ for all elements u . Since cover I is the union of $I(u)$ of all elements u , we know that $|I| = |I(u_1)| + \dots + |I(u_k)|$. Where $u_1 \dots u_k$ are elements chosen in step 4. Therefore we know that $|I| = |I(u_1)| + \dots + |I(u_k)| \leq f + \dots + f = f \cdot |E|$ from the last proof. Therefore $|I| \leq f \cdot |E|$ and from the last part since $|E| \leq |I^*|$, we can see that the f -cover algorithm is an f -approximation for the f -SetCover problem.

- (7 pts) (d) Prove that for every positive integer f , there is an input for which the f -cover algorithm necessarily outputs a cover that is *exactly* f times larger than an optimal one.

Solution: To prove that for every positive integer f , there is an input for which the f -cover algorithm necessarily outputs a cover that is *exactly* f times larger than an optimal one, we can see through an example if this is true. For an instance of the f -SetCover problem if the universe U has $f+1$ elements in it and there are f subsets so that each subset contains one element of the universe exactly and the last subset contains all the elements, then we can see that the optimal solution should include the last subset as it covers all the elements. So the optimal solution is $|I^*| = 1$. To see the f -cover algorithm on this example, we see that the algorithm would select any uncovered element from the universe. After selecting the element in step 4, the algorithm adds all subsets containing that element to the cover. So the algorithm adds at most f subsets to the cover until all elements are covered. In this example, the algorithm selects each element once and adds the corresponding subset to the cover, so the size of $|I|$ is f . Since $|I^*| = 1$ and $|I| = f$, the f -cover algorithm outputs a cover f times larger than the optimal output. Therefore there is an input that exists for every positive integer f .

(5 EC pts) 7. **Optional extra-credit question: disk storage**

You have been hired as a summer intern to work on a disk storage system. There are two identical disks, each with storage capacity of L . There are n files $F = \{f_1, \dots, f_n\}$, where file f_i has size ℓ_i . A file cannot be split between the two disks, it must be stored entirely on one or the other (if it is stored at all).

The Disk Storage Optimization Problem is to store the *maximum number* of files from F on the two disks. It turns out that the decision version of this problem is NP-complete (maybe you can prove it!).

Give a polynomial-time algorithm that produces a solution—i.e., a selection of files and which disk to store each of them on—that stores a number of files *within just one* of optimal.

This result is neat, since all of the approximation algorithms we've seen so far come within some *multiplicative* factor of optimal. Here, the algorithm finds a solution that is only an *additive* amount worse than optimal!

Solution: We can show an algorithm in polynomial time through a greedy algorithm.

- 1) first we can take the files and sort them in a non-increasing order with size of l .
- 2) then we create two counters for each disk to keep track of each remaining capacity
- 3) iterate through the sorted files and try storing each file on both disks, choosing the disk with the larger remaining capacity to put the file on.
- 4) do until the capacities are reached for both disks (or not enough to add another file) or until the files are all processed.
- 5) return the files and the disks that the files are on

We can see that this algorithm is polynomial as the iteration through the files is linear while the sorting from the disks considering the remaining space is logarithmic. Therefore those complexities cover the entire algorithm and the time is polynomial.

we can see that this algorithm is correct. If the optimal solution stores m files in the disks and the total size of the m files is l , we put this through the greedy algorithm. We know that the algorithm puts the files in the disk with larger remaining space, therefore the algorithm has to use the same amount of the disks or less than the optimal solution with m files. Since the algorithm stores larger files first, we have less remaining capacity each turn. Since the optimal solution stores m files, the total size of the m files is at most $2l$ which is the capacity of the two disks. Therefore the greedy algorithm we know will not leave more than l empty space in total. Therefore the files stored by the greedy algorithm should be within $m+1$ of the optimal solution, which is what is desired by the requirement of the algorithm.